



UNIVERSITÀ DI PADOVA

Dipartimento
di Ingegneria
dell'Informazione

Modern Video Compression Standards

Marco Cagnazzo

Multimedia Communications

The Hybrid Video Coding Paradigm

Space partitioning and RDO

Spatial and Temporal Prediction

Transforms, Quantization, and Filters

Network Abstraction and Parallelism

The Hybrid Video Coding Paradigm

Space partitioning and RDO

Spatial and Temporal Prediction

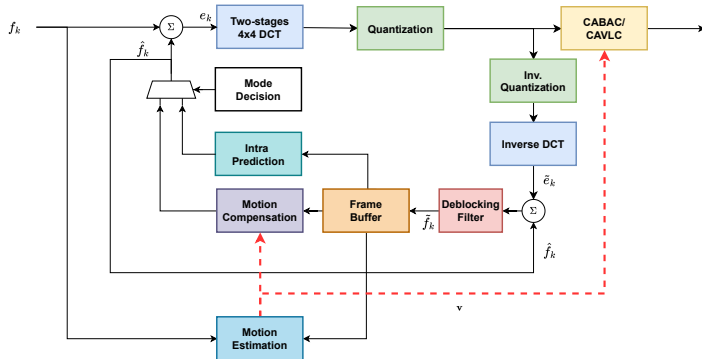
Transforms, Quantization, and Filters

Network Abstraction and Parallelism

The Universal Hybrid Video Encoder

From MPEG-1 to VVC, every standard utilizes this fundamental architecture:

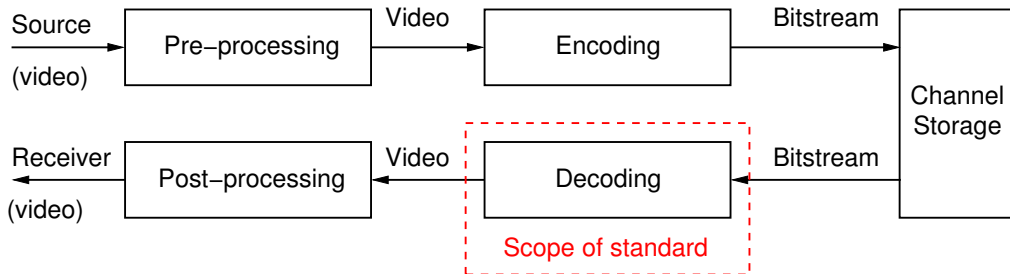
Core processes:



- ▶ Spatial (Intra) and temporal (Inter) prediction
- ▶ Residual calculation
- ▶ Transform and Quantization
- ▶ Internal closed loop (Inverse Quantization/Transform) to reconstruct reference frames and prevent *drift*

Scope of the Standard

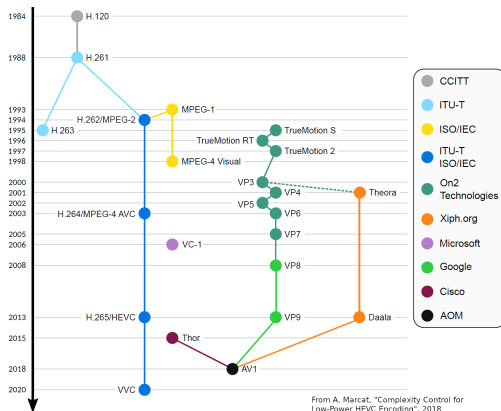
If the fundamental architecture is universal, what exactly is standardized?



Interoperability and Competition

Standards **exclusively** define the bitstream syntax and the decoding process. Encoder architecture and optimization strategies remain open problems for industrial competition.

Codecs Historical Evolution



To improve compression, standards continuously increase the encoder's "degrees of freedom" (partitioning options, prediction modes).

The Golden Rule

Generational leaps typically halve the required bit-rate for identical perceptual quality, resulting in a computational complexity increase of approximately $10\times$ for the encoder.

Market and Use Cases (2026 Scenarios)

Currently, multiple standards coexist. Codec adoption is driven by licensing models and hardware availability rather than strict technical superiority.

Ecosystem	Typical Codecs & Features	Market Share
Real-Time & Legacy (Video calls, old devices)	H.264/AVC Low-latency support, universal HW compatibility.	H.264: ~80–90%
Broadcast & Mobile (DVB-T2, Premium 4K)	H.265/HEVC, H.264/AVC High efficiency, mature HW ecosystem.	HEVC: ~50–70% H.264: ~30–50%
Web Streaming (YouTube, VOD)	AV1, H.264/AVC, H.265/HEVC OTT-optimized, AV1 royalty-free	H.264: ~60% AV1: ~20–30% HEVC: ~10–20%
Next-Gen (8K, 360°, VR)	H.266/VVC, AV1 Maximum efficiency, high computational complexity.	AV1: growing VVC: <5%

Video Codec comparison

Let's observe the empirical consequences of encoder evolution.

Compress the raw *HoneyBee*¹ sequence (1080p, 120fps) forcing a target coding rate of **2.0 Mbps**.

Codec	Actual Rate	PSNR	Enc. Time (s)	Playback
MPEG-2	7.06 Mbps	36.58 dB	2.18	[Play]
H.264	2.00 Mbps	40.51 dB	17.06	[Play]
HEVC	1.98 Mbps	41.69 dB	109.45	[Play]
AV1	1.80 Mbps	43.47 dB	158.66	[Play]
VVC	1.98 Mbps	43.81 dB	105.08	[Play]

¹A. Mercat, M. Viitanen, and J. Vanne, "UVG dataset: 50/120fps 4K sequences for video codec analysis and development," in Proc. ACM Multimedia Syst. Conf., Istanbul, Turkey, June 2020.
Copyright: Digiturk 2013.

Visual Comparison: HoneyBee



MPEG-2

Rate: 7.06 Mbps

PSNR: 36.58 dB

MPEG-2 is not able to attain the target bit-rate. The blocking artifacts are quite visible

Visual Comparison: HoneyBee



H.264

Rate: 2.00 Mbps
PSNR: 40.51 dB

H.264 attains the target bit-rate. The blocking artifact are removed, but some artifacts are still visible.

Visual Comparison: HoneyBee



HEVC

Rate: 1.98 Mbps

PSNR: 41.69 dB

Improved quality

Visual Comparison: HoneyBee



AV1

Rate: 1.80 Mbps

PSNR: 43.47 dB

Top quality

Visual Comparison: HoneyBee



VVC

Rate: 1.98 Mbps

PSNR: 43.81 dB

Top quality

Video Codec comparison

Compress the raw *Bosphorus*² sequence (1080p, 120fps) using a VBV target of **2.0 Mbps**.

Codec	Actual Rate	PSNR	Enc. Time (s)	Playback
MPEG-2	8.47 Mbps	36.14 dB	2.45	[Play]
H.264	2.03 Mbps	35.58 dB	19.20	[Play]
HEVC	1.97 Mbps	38.82 dB	143.83	[Play]
AV1	1.98 Mbps	40.69 dB	147.13	[Play]
VVC	1.96 Mbps	41.00 dB	139.08	[Play]

²A. Mercat, M. Viitanen, and J. Vanne, "UVG dataset: 50/120fps 4K sequences for video codec analysis and development," in Proc. ACM Multimedia Syst. Conf., Istanbul, Turkey, June 2020.
Copyright: Digiturk 2013.

Visual Comparison: Bosphorus



MPEG-2
Rate: 8.47 Mbps
PSNR: 36.14 dB

Visual Comparison: Bosphorus



H.264

Rate: 2.03 Mbps

PSNR: 35.58 dB

Visual Comparison: Bosphorus



HEVC

Rate: 1.97 Mbps

PSNR: 38.82 dB

Visual Comparison: Bosphorus



AV1

Rate: 1.98 Mbps

PSNR: 40.69 dB

Visual Comparison: Bosphorus



VVC

Rate: 1.96 Mbps

PSNR: 41.00 dB

The empirical data shows that newer standards achieve significantly better PSNR at the same bit-rate, but require ever increasing encoding time.

- ▶ **MPEG-2 Underflow:** The encoder reaches its maximum quantization limit. It lacks the tools to discard further bits, resulting in severe coding rate target overshoots (7–8 Mbps).
- ▶ **The RDO Challenge:** VVC and AV1 have a massive tree of potential block partitions and prediction modes. Finding the optimal configuration to stay under 2.0 Mbps while maximizing quality requires evaluating millions of combinations.

How does the encoder make these decisions?

→ *Rate-Distortion Optimization*

The Hybrid Video Coding Paradigm

Space partitioning and RDO

Spatial and Temporal Prediction

Transforms, Quantization, and Filters

Network Abstraction and Parallelism

Recall: Rate-Distortion Optimization

From the previous lecture, we know that the encoder minimizes the Lagrangian cost function for each block:

$$J = D + \lambda R \quad (1)$$

The Block Partitioning Problem:

- ▶ Partitioning a block into sub-blocks is treated as a distinct coding mode
- ▶ The encoder evaluates the cost of the original block (J_B) against the sum of the sub-blocks ($J_{split} = \sum J_{subblock}$)
- ▶ If $J_{split} < J_B$, the split is accepted
- ▶ *The evolution of video standards is largely the evolution of allowable split geometries.*

H.264/AVC: The Macroblock Era

H.264 relies on a fixed rigid structure, which limits the RDO search space

- ▶ **Base Unit:** The 16×16 Macroblock (MB)
- ▶ **Variable Block-Size (VBS):** The MB can be partitioned into 16×8 , 8×16 , or 8×8 blocks to better capture motion boundaries
- ▶ **Sub-partitions:** 8×8 blocks can be further divided down to 4×4

Limitation

A maximum size of 16×16 is highly inefficient for High Definition (1080p) or 4K video, where smooth background areas (like sky or walls) span hundreds of pixels.

H.265/HEVC: The Quad-Tree Paradigm

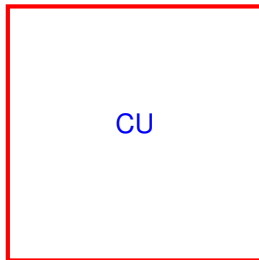
HEVC abandons Macroblocks in favor of the **Coding Tree Unit (CTU)** to handle UHD resolutions

Larger Base Size: CTUs can be up to 64×64 pixels

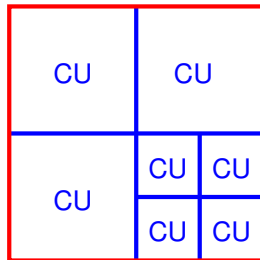
Quad-Tree Split: A CTU is recursively split into 4 square Coding Units (CU), down to 4×4 CUs

Functional Separation:

CTU



CTU



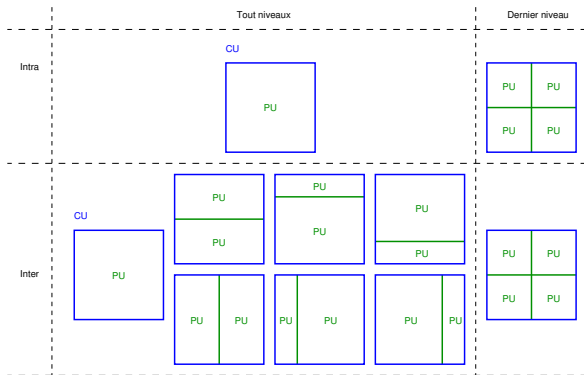
H.265/HEVC: The Quad-Tree Paradigm

HEVC abandons Macroblocks in favor of the **Coding Tree Unit (CTU)** to handle UHD resolutions

Larger Base Size: CTUs can be up to 64×64 pixels

Quad-Tree Split: A CTU is recursively split into 4 square Coding Units (CU), down to 4×4 CUs

Functional Separation: A CU can be symmetrically or asymmetrically divided into Prediction Units (PU)



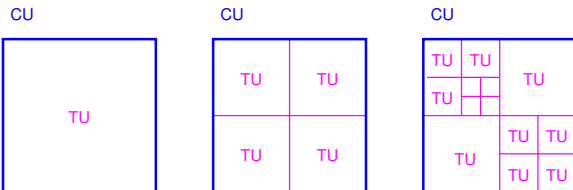
H.265/HEVC: The Quad-Tree Paradigm

HEVC abandons Macroblocks in favor of the **Coding Tree Unit (CTU)** to handle UHD resolutions

Larger Base Size: CTUs can be up to 64×64 pixels

Quad-Tree Split: A CTU is recursively split into 4 square Coding Units (CU), down to 4×4 CUs

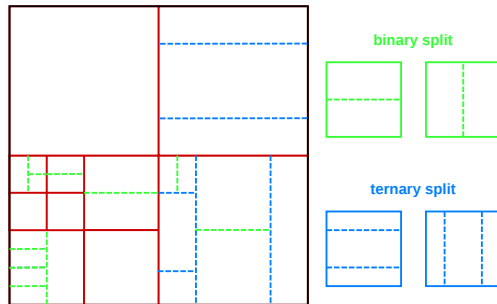
Functional Separation: A CU can be symmetrically or asymmetrically divided into Prediction Units (PU) and Transform Units (TU)



H.266/VVC: QT + Multi-Type Tree (MTT)

VVC introduces extreme flexibility, eliminating the strict separation between CUs, PUs, and TUs for most modes

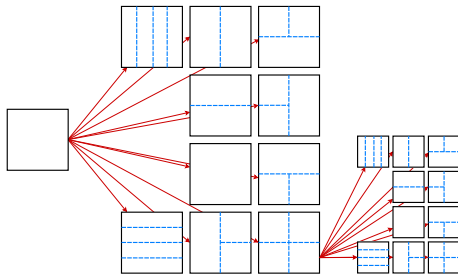
- ▶ **Massive Base Size:** CTUs up to 128×128 pixels
- ▶ **QT + MTT:** Quad-Tree leaves can be further partitioned using a Multi-Type Tree
- ▶ **New Splits:** Includes *Binary* (symmetric half) and *Ternary* (1:2:1 ratio) splits
- ▶ Rectangular blocks become fundamental for encoding object edges accurately.



AV1: 10-Way Superblock Partitioning

AV1 utilizes a similar highly flexible hierarchical approach

- ▶ **Base Unit:** The *Superblock*, sized up to 128×128
- ▶ **10-Way Split:** A block can be partitioned using 10 different geometrical patterns
- ▶ Includes 4:1 and 1:4 rectangular splits, creating highly elongated blocks for sharp directional textures.
- ▶ Square blocks can be recursively partitioned



The Algorithmic Burden of RDO

The Decision Tree

To find the optimal partition, the encoder explores the block geometry using a **Depth-First Search (DFS)** algorithm, optimizing the cost $J = D + \lambda R$.

The evaluation is a massive sequential process:

1. **Top-Down Exploration:** Try coding the block as a whole. Then, try splitting it.
2. **Causal Dependency:** Sub-blocks are tested sequentially (Z-scan order). A block cannot be predicted until top and left blocks are fully *reconstructed* (pixels are frozen).
3. **Inner Loop (Prediction):** For each block, exhaustively test Intra/Inter modes to find its local minimum J_{CU} .
4. **Bottom-Up Decision:** Compare the total cost: if $\sum J_{children} < J_{parent}$, the split is kept.

Conclusion: While the theoretical state space of a CTU is enormous, reference models traverse it sequentially. Commercial encoders drastically cut this process using **Fast Heuristics and Machine Learning** to prune the tree early.

Pruning the Decision Tree: Fast Heuristics

To mitigate the extreme complexity of RDO, production encoders (e.g., VVenC, SVT-AV1) implement aggressive early termination strategies.

Common Pruning Approaches:

- ▶ **Spatial/Texture Analysis:** Evaluating local variance and directional gradients. If a 64×64 Coding Unit exhibits low variance (e.g., flat background), deeper QT/MTT partitions are immediately bypassed.
- ▶ **Temporal Correlation:** Constraining the RDO search space by exploiting the partition depth of the co-located CU in the reference frame.
- ▶ **Machine Learning Classifiers:** Deploying lightweight CNNs or gradient boosting decision trees (e.g., LightGBM) to predict split probabilities directly from raw pixel features, discarding highly improbable geometries before any RDO calculation.

The Hybrid Video Coding Paradigm

Space partitioning and RDO

Spatial and Temporal Prediction

Transforms, Quantization, and Filters

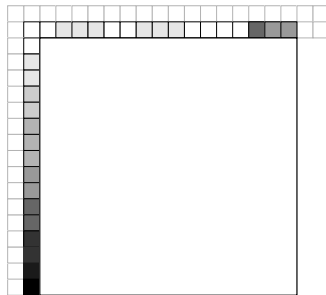
Network Abstraction and Parallelism

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

Number of predictors vs. signalization cost: How many bits B are needed to signal the H.264 Intra mode? How many different predictors P can be generated?



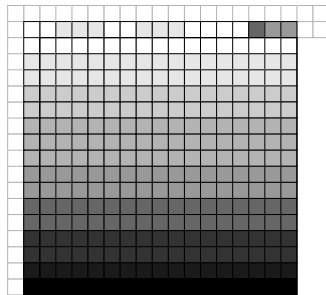
H.264

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



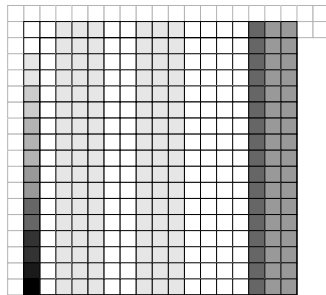
H.264 16×16 Intra modes
2 bits, 4 predictors

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



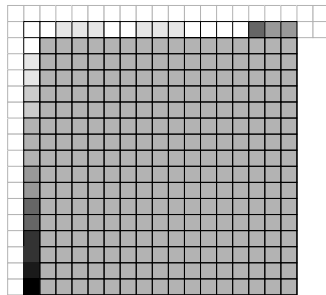
H.264 16×16 Intra modes
2 bits, 4 predictors

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



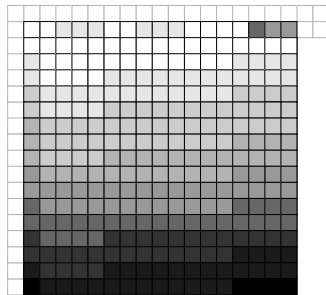
H.264 16×16 Intra modes
2 bits, 4 predictors

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



H.264 16×16 Intra modes
2 bits, 4 predictors

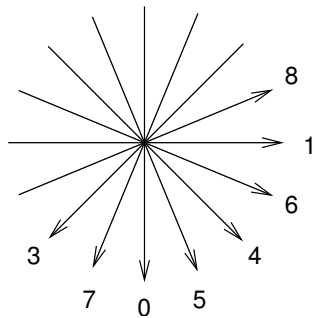
Intra Prediction

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



H.264 4×4 Intra modes
 1-4 bits per subblock,
 $\approx 2 \cdot 10^{15}$ predictors

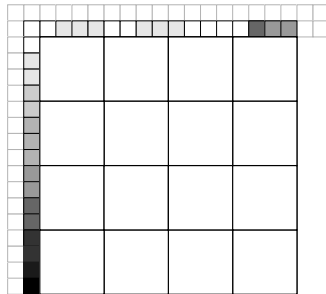
Intra Prediction

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



H.264 4×4 Intra modes
 1-4 bits per subblock,
 $\approx 2 \cdot 10^{15}$ predictors

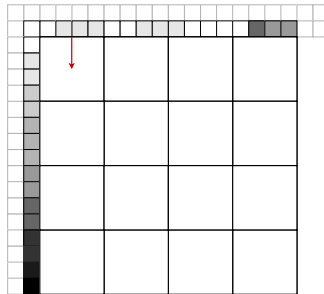
Intra Prediction

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



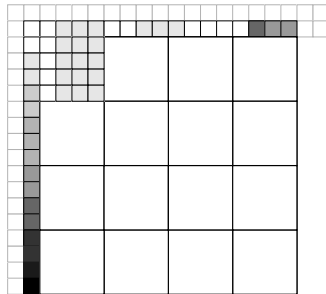
H.264 4×4 Intra modes
 1-4 bits per subblock,
 $\approx 2 \cdot 10^{15}$ predictors

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



H.264 4×4 Intra modes
1-4 bits per subblock,
 $\approx 2 \cdot 10^{15}$ predictors

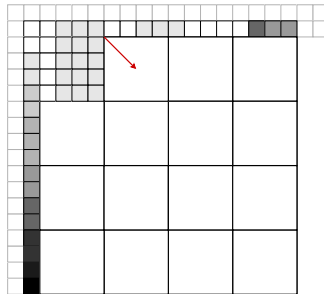
Intra Prediction

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



H.264 4×4 Intra modes
 1-4 bits per subblock,
 $\approx 2 \cdot 10^{15}$ predictors

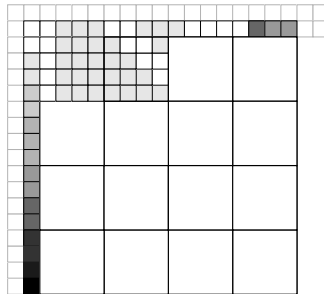
Intra Prediction

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



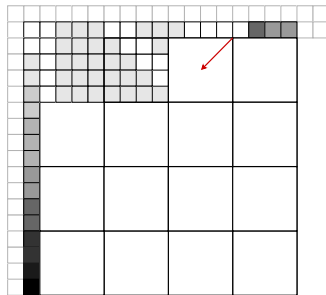
H.264 4×4 Intra modes
 1-4 bits per subblock,
 $\approx 2 \cdot 10^{15}$ predictors

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



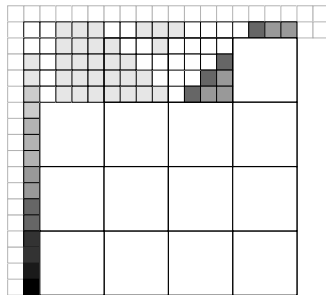
H.264 4×4 Intra modes
1-4 bits per subblock,
 $\approx 2 \cdot 10^{15}$ predictors

Intra prediction exploits spatial redundancy by extrapolating pixels from already-decoded neighboring blocks.

The Evolution of Freedom:

- ▶ **H.264/AVC:** 4 prediction types for 16×16 blocks, and 9 types (8 directions + DC) for 4×4
- ▶ **H.265/HEVC:** Expands to 35 modes (33 directions + DC + Planar) to handle complex textures
- ▶ **H.266/VVC:** Explodes to 65 directional modes, plus wide-angle modes for non-square blocks

$$P \propto 2^B$$



H.264 4×4 Intra modes
1-4 bits per subblock,
 $\approx 2 \cdot 10^{15}$ predictors

The Cost of Freedom: Predicting the Predictor

The Overhead Problem

If VVC has 67 intra modes, sending the mode index requires ≈ 7 bits per block. For tiny 4×4 blocks, this signaling overhead would completely destroy the compression gains.

Solution: Most Probable Mode (MPM)

- ▶ The encoder predicts the mode itself using the spatial context
- ▶ A list of Most Probable Modes (MPM) is built using the directions chosen by the Top and Left neighbors
- ▶ HEVC uses a 3-candidate MPM list; VVC expands this to a 6-candidate list
- ▶ If the chosen mode is in the MPM list, it is coded with a very short index (e.g., 1 or 2 bits)

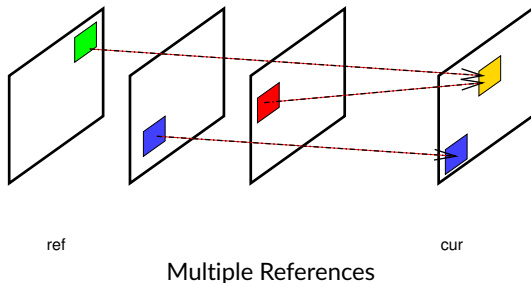
Inter Prediction: Fractional Motion Compensation

Inter prediction removes temporal redundancy via Motion Estimation and Compensation (ME/MC)

- ▶ Objects rarely move by exact integer pixel amounts between frames.
- ▶ **H.264/HEVC/VVC**: Utilize quarter-pel ($1/4$) fractional precision The encoder generates sub-pixels using interpolation filters (e.g., 6-tap filters in H.264)
- ▶ **AV1**: Pushes the precision even further to $1/8$ th pixel accuracy, utilizing dynamically selectable interpolation filters (e.g., sharp, smooth)

Modern standards generalize the concept of temporal distance. A frame is no longer restricted to predicting from its immediate chronological neighbor.

- ▶ **Multiple Reference Frames:** Slices can maintain one (P) or two (B) lists of reference images kept in the Decoded Picture Buffer (DPB)
- ▶ **Generalized B-Slices:** The encoder can flexibly select predictors from these lists and combine them using weighted averages



Vector Prediction: From Skip to Merge

Just like Intra modes, Motion Vectors (MVs) must be predicted to save bits.

- ▶ **H.264 (Skip/Direct):** Uses a simple median of spatial neighbors. If the median is correct, 0 bits are spent on MVs
- ▶ **HEVC/VVC (Merge Mode):** Replaces the median with a dynamic *candidate list*
 - ▶ The list aggregates 5 spatial candidates (adjacent blocks) and 1 temporal candidate (co-located block in the reference frame)
 - ▶ The encoder simply transmits the index of the best candidate in the list, sharing both the MV and the reference frame index

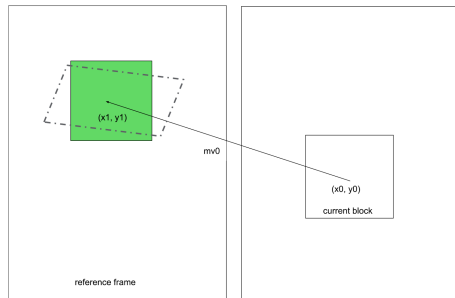
To squeeze the final bits of temporal redundancy out of the signal, AV1 and VVC abandon the assumption that blocks only move rigidly in 2D space.

Affine Motion Prediction:

- ▶ Allows blocks to rotate, zoom, and shear.
- ▶ Uses 4-parameter or 6-parameter models instead of a simple (x, y) vector.

Compound Prediction:

- ▶ AV1 allows combining Inter and Intra predictions within the same block



AV1 Affine Prediction

The Hybrid Video Coding Paradigm

Space partitioning and RDO

Spatial and Temporal Prediction

Transforms, Quantization, and Filters

Network Abstraction and Parallelism

Residual Coding: The Evolution of Transforms

After prediction, the residual error must be decorrelated into the frequency domain.

The push for larger and flexible transforms:

- ▶ **H.264/AVC:** Relies on a 4×4 integer-coefficient approximation of the DCT, perfectly synchronized to avoid precision errors between encoder and decoder
- ▶ **H.265/HEVC:** Expands transforms up to 32×32 to handle 4K resolutions It also introduces a DST (Discrete Sine Transform) approximation specifically for 4×4 Intra blocks, improving compression by $\approx 1\%$
- ▶ **H.266/VVC:** Introduces **Multiple Transform Selection (MTS)** and non-square transforms to perfectly match the rectangular partitions created by the QT+MTT structure

Quantization and Entropy Coding

Quantization is the only lossy step in the hybrid encoder.

Quantization (QP):

- ▶ In H.264/HEVC, the quantization step size doubles for every increment of 6 in the Quantization Parameter (QP)
- ▶ Empirically, a unitary QP increment increases the rate by $\approx 12.5\%$

Lossless Entropy Coding:

- ▶ Standardized entirely around **CABAC** (Context-Adaptive Binary Arithmetic Coding).
- ▶ CABAC transforms syntax elements into binary formats and adapts probabilities dynamically based on previously decoded spatial contexts
- ▶ It yields a 5% – 15% rate reduction over older variable-length codes (like CAVLC)

The Need for In-Loop Filtering

Because the transform and quantization are applied independently to disjoint blocks, strong discontinuities (blocking artifacts) appear at block boundaries at low bit-rates

Reference image



Reference Image

Current image



Current Image

Motion-compensated image



Motion-Compensated (Artifacts)

If left uncorrected, these artifacts are saved in the Decoded Picture Buffer (DPB) and will permanently ruin future motion compensations

To fix quantization damage, standards mandate complex normative filters inside the reconstruction loop

- ▶ **H.264/AVC (Deblocking):** Analyzes edges between 4×4 blocks. Filtering strength adapts dynamically to the coding mode, motion vectors, and quantization step of neighboring blocks
- ▶ **H.265/HEVC (Deblocking + SAO):** Applies deblocking only on an 8×8 grid to reduce complexity. Introduces the **Sample Adaptive Offset (SAO)**, which categorizes pixels (edges, bands) and adds offsets to correct ringing artifacts

As transform sizes grew and bit-rates plummeted, deblocking and SAO were no longer sufficient. Modern codecs dedicate massive computational power to pixel restoration.

- ▶ **H.266/VVC:** Adds the **Adaptive Loop Filter (ALF)** and **Luma Mapping with Chroma Scaling (LMCS)** specifically designed to preserve details in HDR (High Dynamic Range) video
- ▶ **AV1:** Implements a formidable post-processing pipeline :
 - ▶ *Directional Filters (CDEF):* Analyzes gradient directions to smooth edges without blurring textures
 - ▶ *Loop Restoration Filters:* Wiener filters applied over large windows
 - ▶ *Film Grain Synthesis:* Denoises the image before encoding, and synthesizes artificial grain at the decoder to recreate the cinematic look at zero bitrate cost

The Hybrid Video Coding Paradigm

Space partitioning and RDO

Spatial and Temporal Prediction

Transforms, Quantization, and Filters

Network Abstraction and Parallelism

Transmission and Network Errors

The Fragility of Compression

We have compressed a 4K video into a perfect 2 Mbps mathematical stream. But what happens if we transmit this raw bitstream and **just a few bits** are flipped?

[Play Demo: The Corrupted Bitstream]

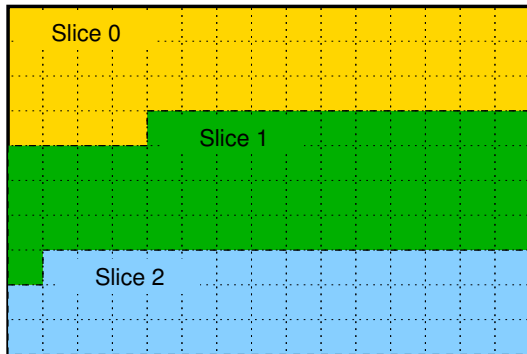
Different "Pipes", Same Vulnerability:

- ▶ **DVB-T (Broadcast):** Continuous stream subject to electromagnetic noise.
- ▶ **Internet (RTP/UDP or TCP/DASH):** Cut into packets (MTU ~ 1500 bytes) that can arrive late or get lost.
- ▶ **Disk Storage (MP4):** Requires specific indexes to seek.

Consequence: CABAC loses context. Errors propagate spatially and temporally via Motion Compensation, creating a "green and gray mosaic".

Pictures and slices

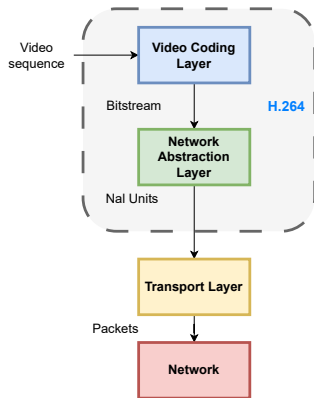
To limit the error propagation, pictures can be divided into **slices**.
At each new slice, INTRA prediction is stopped and CABAC is reset, such that parallel encoding/decoding of slices is possible.



The Solution: Separation of Concerns

Starting from H.264, video standards separate **compression** from **transport**.

The Factory Analogy:

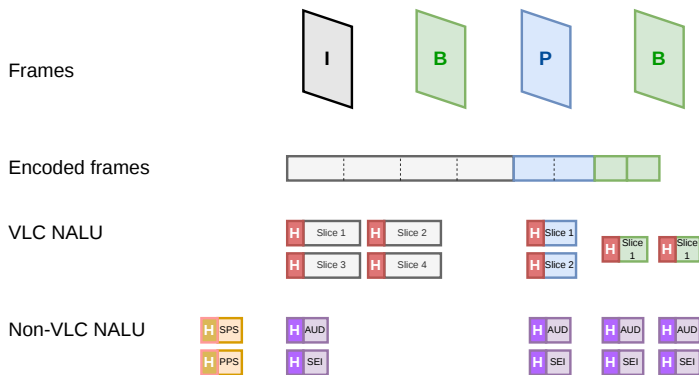


- ▶ **Video Coding Layer (VCL):** The production department. It handles pure compression (Transforms, Prediction, RDO). It generates the *slices* of each compressed picture.
- ▶ **Network Abstraction Layer (NAL):** The shipping department. It takes VCL slices, encapsulates them, and adds metadata.
- ▶ **The Goal:** Provide a standard interface to map the video over RTP/IP (real-time), ISO MP4 (storage), or MPEG-2TS (diffusion).

The VCL encodes each picture into one or more slices. The NAL encapsulates the slices into **VCL NALUs** (one per slice) and generates **Non-VCL NALUs** (SPS, PPS, SEI).

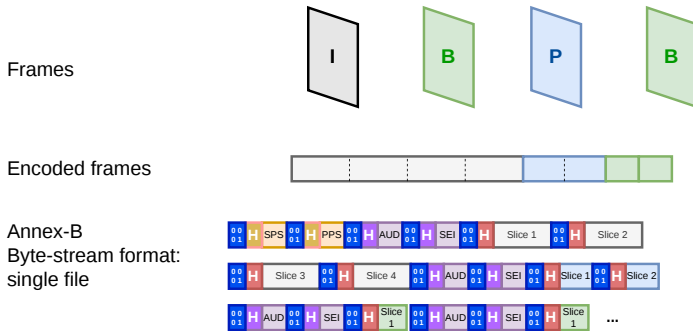
- VCL NALU** contains modes, MVs and quantized coefficients of a slice. It can be **Instantaneous Decoding Refresh** (i.e., Intra with CABAC reset) or non-IDR
- SPS** Sequence parameter set: a non-VCL NALU with important encoding parameters: picture size, color depth, etc.
- PPS** Picture parameter set: per-picture coding options (specific to each codec)
- SEI** Supplemental Enhancement Information: not necessary for decoding, but useful for some specific task (e.g., subtitles, HDR info, user data, etc.)
- AUD** Access Unit delimiter: optional NALU signaling a new picture

NALUs and Access Units



Access Unit (AU): A logical grouping of all NALUs required to decode a single frame. The decoder identifies its boundaries implicitly. How is this sequence of NALUs actually delivered? If the transport medium is a continuous "blind" tube (e.g., MPEG-TS, raw .264 files), we use the **Byte-Stream format**. If the transport layer already provides boundaries (e.g., an MP4 container), we use the **Packetized format**.

Format 1: Annex-B (Byte-Stream)



Mechanism: A Start Code

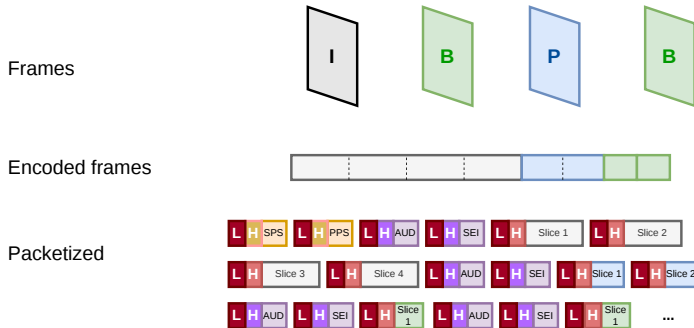
(e.g., 0x00000001) is injected before every NALU.

Emulation Prevention: Fake bytes are injected into the payload to prevent valid video data from accidentally mimicking a Start Code.

Pro: Perfect for continuous streaming or broadcasting where packet boundaries do not exist natively.

Con: CPU overhead for emulation prevention and re-synchronization. The decoder must scan every single byte sequentially to find the next Start Code.

Format 2: Packetized (Length-Prefixed)



Mechanism: A Length Field (usually 4 bytes) is written exactly before the NALU header. **No Start Codes:** Emulation prevention is not strictly required for boundary detection. **Pro:** Highly software-friendly. The parser reads the length and performs an $O(1)$ memory jump directly to the next NALU. **Con:** Useless if the stream is corrupted or the external container is damaged (loss of synchronization).

The Evolution: HEVC, VVC, and AV1

H.265/HEVC and H.266/VVC:

- ▶ Maintain the Annex-B / Packetized dualism.
- ▶ The NALU header expands from 1 to **2 bytes** to include *Temporal ID* and *Layer ID* for spatial/temporal scalability.

AV1: Breaking the Paradigm

- ▶ AV1 abandons the MPEG legacy. NALUs are called **OBUs (Open Bitstream Units)**.
- ▶ **No Start Codes:** AV1 completely drops Annex-B and emulation prevention to save CPU cycles.
- ▶ It uses exclusively a Length-Prefixed approach even for raw bitstreams.

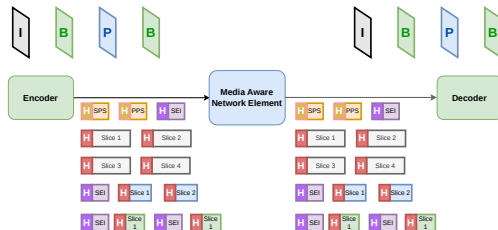
How the NAL enables intelligent network operations without decoding: The NAL header and structure allow intermediate nodes (MANEs) to understand and manage the video stream.

1. Router Inspection (MANE):

- ▶ Media Aware Network Elements (MANEs) can read the 1 or 2-byte header: this is a form of *Deep Packet Inspection*
- ▶ Smart Dropping: If the network is congested, the router can selectively drop Non-Reference B-frames without touching critical IDR frames or Parameter Sets.
- ▶ Add protection (CRC) to sensitive data (SPS)

2. RTP Transport and Fragmentation:

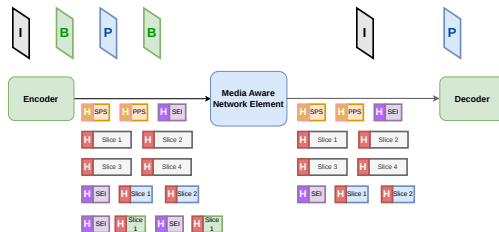
- ▶ If a VCL NALU (e.g., an entire I-frame slice) exceeds the network MTU (1500 bytes), it cannot be transmitted as-is.
- ▶ The RTP layer slices large NALUs into **Fragmentation Units (FUs)** and reassembles them at the receiver, keeping the video decoder entirely unaware of the network constraints.



How the NAL enables intelligent network operations without decoding: The NAL header and structure allow intermediate nodes (MANEs) to understand and manage the video stream.

1. Router Inspection (MANE):

- ▶ Media Aware Network Elements (MANEs) can read the 1 or 2-byte header: this is a form of *Deep Packet Inspection*
- ▶ **Smart Dropping:** If the network is congested, the router can selectively drop Non-Reference B-frames without touching critical IDR frames or Parameter Sets.
- ▶ Add protection (CRC) to sensitive data (SPS)



2. RTP Transport and Fragmentation:

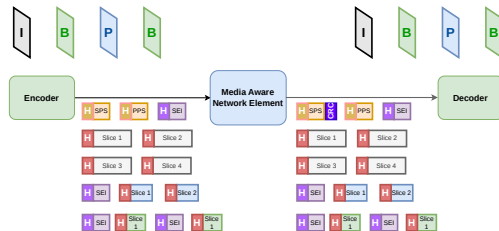
- ▶ If a VCL NALU (e.g., an entire I-frame slice) exceeds the network MTU (1500 bytes), it cannot be transmitted as-is.
- ▶ The RTP layer slices large NALUs into **Fragmentation Units (FUs)** and reassembles them at the receiver, keeping the video decoder entirely unaware of the network constraints.

Network Friendliness in Action

How the NAL enables intelligent network operations without decoding: The NAL header and structure allow intermediate nodes (MANEs) to understand and manage the video stream.

1. Router Inspection (MANE):

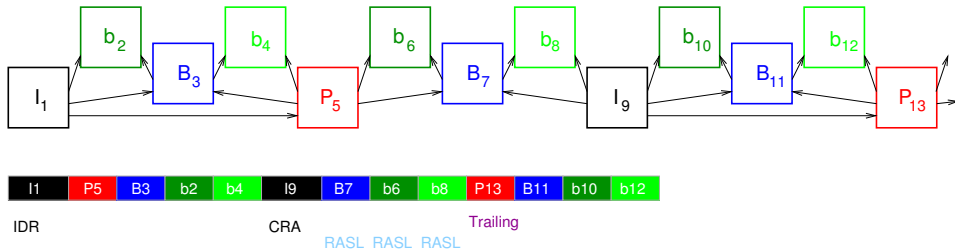
- ▶ Media Aware Network Elements (MANEs) can read the 1 or 2-byte header: this is a form of *Deep Packet Inspection*
- ▶ Smart Dropping: If the network is congested, the router can selectively drop Non-Reference B-frames without touching critical IDR frames or Parameter Sets.
- ▶ Add protection (CRC) to sensitive data (SPS)



2. RTP Transport and Fragmentation:

- ▶ If a VCL NALU (e.g., an entire I-frame slice) exceeds the network MTU (1500 bytes), it cannot be transmitted as-is.
- ▶ The RTP layer slices large NALUs into **Fragmentation Units (FUs)** and reassembles them at the receiver, keeping the video decoder entirely unaware of the network constraints.

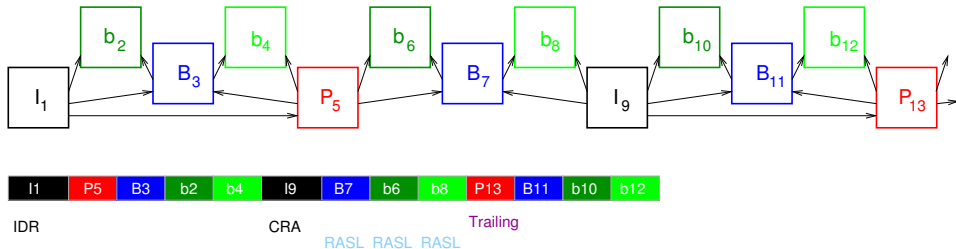
Resynchronization: Stream Structure



Decoding may restart at a **Random Access Point (RAP)**:

- ▶ **IDR (Instantaneous Decoding Refresh)**: A strict barrier that completely clears the Decoded Picture Buffer (DPB). No pictures following the IDR in decoding order can reference pictures preceding it.
- ▶ **Pros**: Guarantees a clean restart. Ideal for the absolute beginning of a video sequence or hard scene cuts.
- ▶ **Cons**: Breaks temporal prediction across the barrier, reducing compression efficiency. It cannot exploit leading pictures for better referencing.

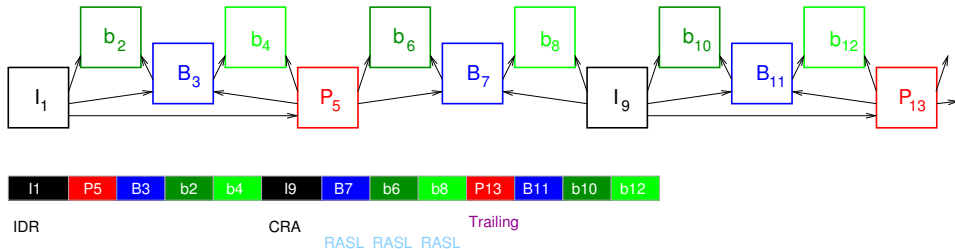
Resynchronization: Stream Structure



Decoding may restart at a **Random Access Point (RAP)**:

- ▶ **CRA (Clean Random Access)**: Introduced in HEVC. An Intra picture that allows **RASL** (Random Access Skipped Leading) frames to reference pictures located *before* the CRA.
- ▶ **Scenario 1 (Normal Playback)**: The decoder has past frames in memory. RASL pictures are fully decoded, maximizing compression efficiency.
- ▶ **Scenario 2 (Seeking)**: Decoding starts directly at the CRA. Past references are missing, so the decoder safely discards (skips) the RASL pictures and decodes the rest cleanly.
- ▶ **Comparison**: H.264 lacked this feature, forcing the frequent use of costly IDR frames

Resynchronization: Stream Structure

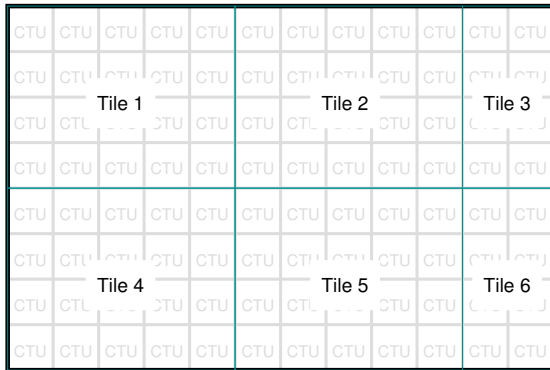


Decoding may restart at a **Random Access Point (RAP)**:

- ▶ **BLA (Broken Link Access):** Functions similarly to a CRA, but explicitly signals that the required reference pictures for its leading frames are destroyed.
- ▶ **Origin:** It is typically generated by a **secondary encoder in network nodes** (e.g., transcoders, packagers, ad-insertion splicers) rather than the original content encoder.
- ▶ **Mechanism:** When a stream is cut and spliced at a CRA, the network node alters the NALU header from CRA to BLA. The decoder reads this and immediately drops the broken RASL pictures without attempting reference lookups.

Hardware Friendliness: Tiles

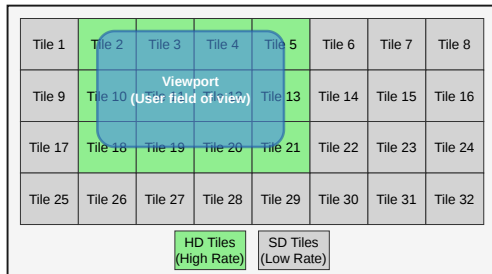
- ▶ The CTU may be grouped into rectangular regions referred to as “tiles”.
- ▶ The tiles are “self-contained” and independent from slices.
- ▶ A slice may contain several tiles, or a tile may contain several slices.
- ▶ One of the main targets of tiles is to allow **parallel encoding and decoding** across multi-core processors.
- ▶ Another application: viewport priority in 360 video



Tiles in 360-Degree Video

Tiles are the foundational technology for **Viewport-Adaptive Streaming** in Virtual Reality (MPEG-OMAF).

- ▶ **The Problem:** A VR headset only displays $\sim 15\%$ of the full 360 sphere (Field of View). Transmitting the unseen 85% at 8K is a massive waste of bandwidth.
- ▶ **The Tile Solution:** The sphere is projected and encoded as a grid of independent HEVC Tiles.
- ▶ **Dynamic Stitching:** The client requests only the Tiles within the current Viewport at High Resolution, and the background Tiles at very Low Resolution (as a fallback).
- ▶ **Why Tiles?** Because motion vectors cannot cross Tile boundaries, making them perfectly independent to mix and match qualities.



Bandwidth Savings: By using Tile-based adaptive streaming, bandwidth consumption can be reduced by 50-60% without degrading the user's perceived quality.

-
- Diagram illustrating Wavefront Parallelism. A 5x12 grid of CTU (Coding Tree Unit) blocks is shown. The top row is labeled "Wavefront" in purple. The grid is divided into five horizontal bands, each representing a thread. Thread 1 (red) processes the first row. Thread 2 (orange) processes the second row. Thread 3 (yellow) processes the third row. Thread 4 (green) processes the fourth row. Thread 5 (blue) processes the fifth row. Arrows indicate the direction of processing from left to right. The grid is filled with "CTU" labels, and the threads are labeled "Thread 1" through "Thread 5".

Summary of the Hybrid Architecture

The cost of compression:

- ▶ We moved from fixed 16×16 blocks to extreme QT+MTT geometries (128×128).
- ▶ We expanded Intra directions from 9 to 67 and introduced Sub-Pixel Affine motion.
- ▶ We deployed complex in-loop filters (ALF, Restoration) to mask quantization artifacts.
- ▶ We engineered NALUs, mappings, Tiles, and WPP to make it transportable and parallelizable.

The Next Frontier

The hybrid paradigm has arguably reached its mathematical asymptote with VVC. The next major leap requires abandoning discrete mathematical transforms in favor of end-to-end differentiable neural architectures.

→ *Next Lectures: Neural Video Coding*